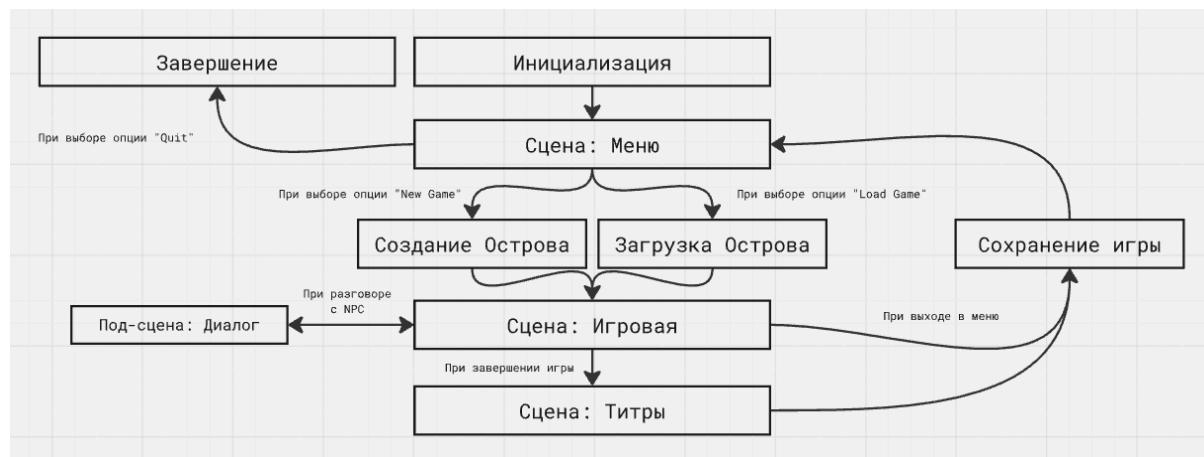


Архитектурный Документ

Глобальная Архитектура

Диаграмма состояний игры:



Инициализация:

В этот момент происходит загрузка всех ресурсов из файлов, а также создаются глобальные объекты. Автоматически переводит состояние на сцену меню.

Завершение:

Все ресурсы освобождаются, игра закрывается.

Сцена Меню:

На этой сцене у игрока есть выбор для загрузки и создания острова, а также возможность выйти из игры.

Создание/Загрузка острова:

Остров либо генерируется либо загружается из файла. После чего создаются объекты связанные с островом, инициализируются и запускаются все геймплейные системы. По завершению переводит на саму сцену игры.

Сцена Игровая:

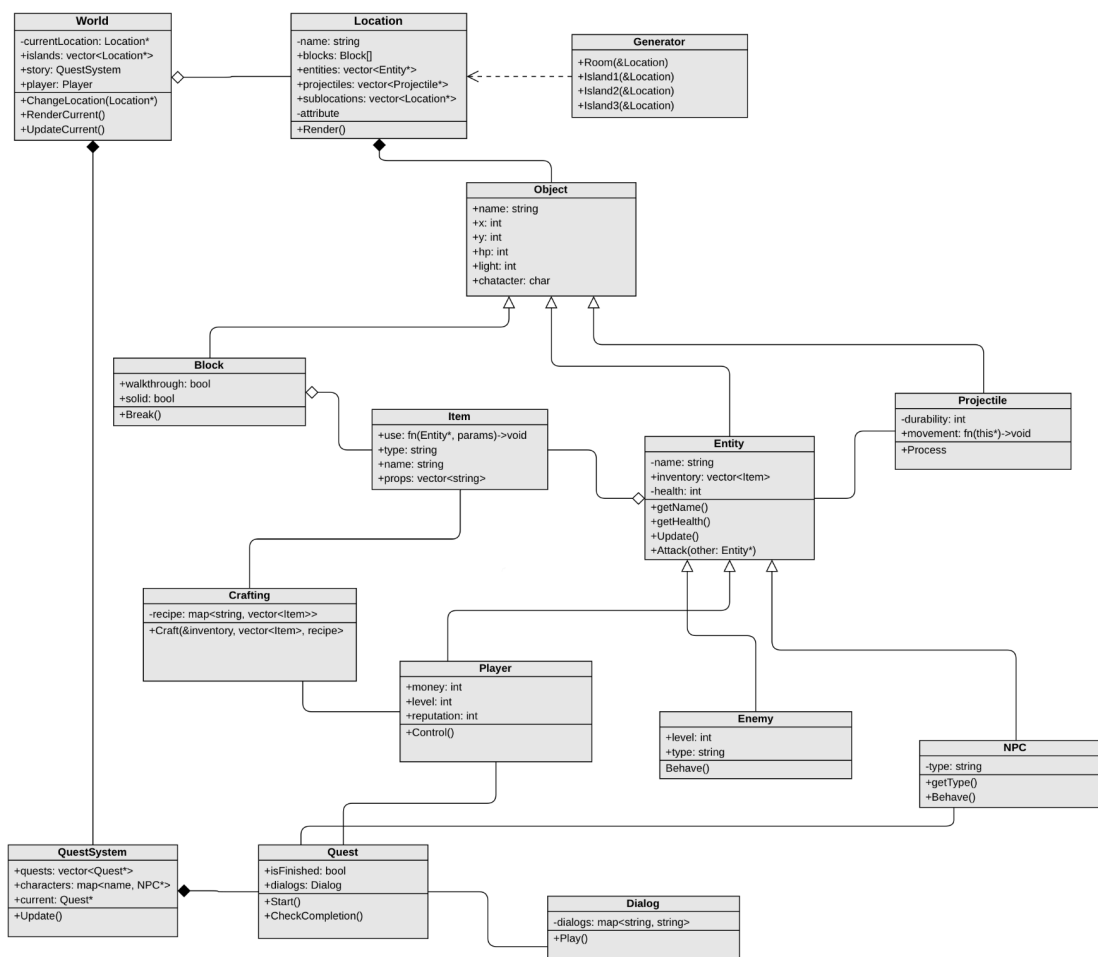
Здесь происходит всё мясо. При разговоре с NPC открывается подсцена диалога, при котором все обновления игры прекращаются. При выходе в меню все данные игровой сессии сохраняются в файл. При завершении с концовкой переходит на титры.

Сцена Титров:

Показывается кто работал над игрой и дает возможность выйти в меню, при этом сохраняя игру.

Классовый Анализ

Классовая Диаграмма:



World

Этот класс по сути связывает все данные игровой сессии в себе.

```
-currentLocation: Location*
+islands: vector<Location*>
+story: QuestSystem
+player: Player
```

+ChangeLocation(Location*) +RenderCurrent() +UpdateCurrent()
--

Location

Класс, который представляет собой локацию, содержит в себе блоки, существа и projectiles
--

-name: string +blocks: Block[] +entities: vector<Entity*> +projectiles: vector<Projectile*> +sublocations: vector<Location*>
--

+Render() +Update()

Generator

Генерирует разные типы локаций

-

+Room(&Location) +Island1(&Location) +Island2(&Location) +Island3(&Location)

Object

Класс, который хранит информацию об объекте

+ name: string + x: int + y: int + hp: int + light: int + character: char
--

Block
Класс, который характеризует блоки в игре
+ walkthrough: bool + solid: bool
+ Break()

Item
Класс, хранящий информацию о вещах в игре
+ use: fn(Entity*, params) -> void + type: string + name: string + props: vector<string>

Entity
Класс, который характеризует все живые существа в игре
- name: string + inventory: vector<Item>
+getName() +getHealth +Update() +Attack(other: Entity*)

Player
Класс о главном герое
+ money: int + level: int + reputation: int
+ Control()

Enemy
Класс о недоброжелательных мобах
+ level: int + type: string
+ Behave()

NPC
Класс об остальных мобах, что присутствуют в игре
- type: string
+ getType() + Behave()

Projectile
Класс, характеризующий снаряды в игре
- durability: int + movement: fn(this*) -> void
+ Process()

Crafting
Класс о процессе создания новых объектов в игре
- recipes: map<string, vector<Item>>
+ Craft(&inventory: vector<Item>, recipe>

Quest
Класс, который описывает события одного квеста в рамках игры
+ isFinished: bool

+ dialogs: Dialog
+ Start() + CheckCompletion()

QuestSystem
Класс, который связывает все квесты между собой
+ quests: vector<Quest*> + characters: map<name, NPC*> + current: Quest*
+ Update()

Dialog
Класс, описывающий диалоги в игре
- dialogs: map<string, string>
+ Play()

Механики и Сценарии

Механики:

Path-finding:

Реализуется через алгоритм A*

Освещение:

Также реализуется через алгоритм BFS, все ячейки, куда можно попасть за определенное кол-во шагов от источника освещения, подсвечиваются, а куда попасть нельзя, те отображаются черным.

Крафт:

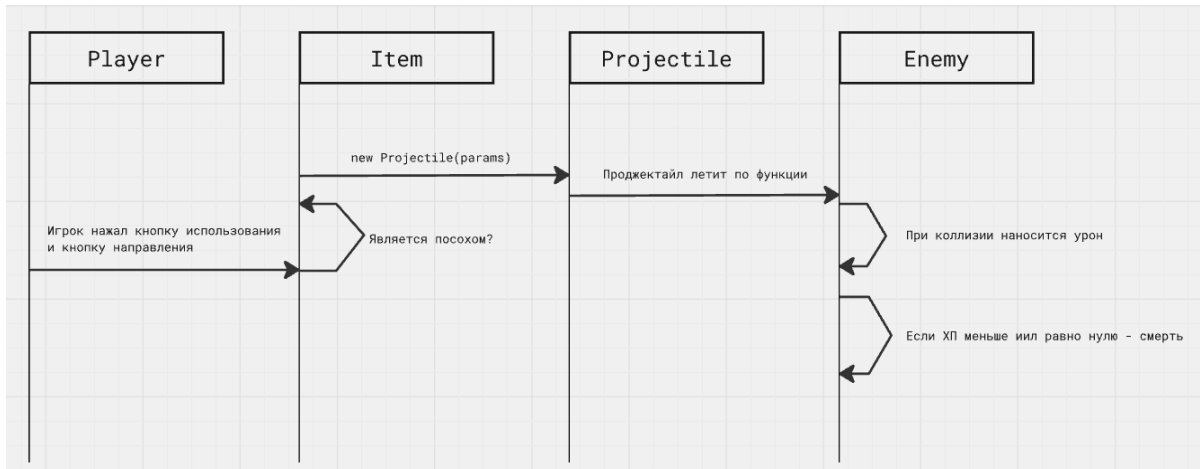
1. Игрок выбирает предмет для крафта в интерфейсе
2. Происходит поиск по рецептам для получения материалов для крафта
 - a. Если у игрока есть необходимые предметы, то они забираются и игрок получает этот самый предмет
 - b. Если у игрока нет предметов, ничего не происходит

Генерация:

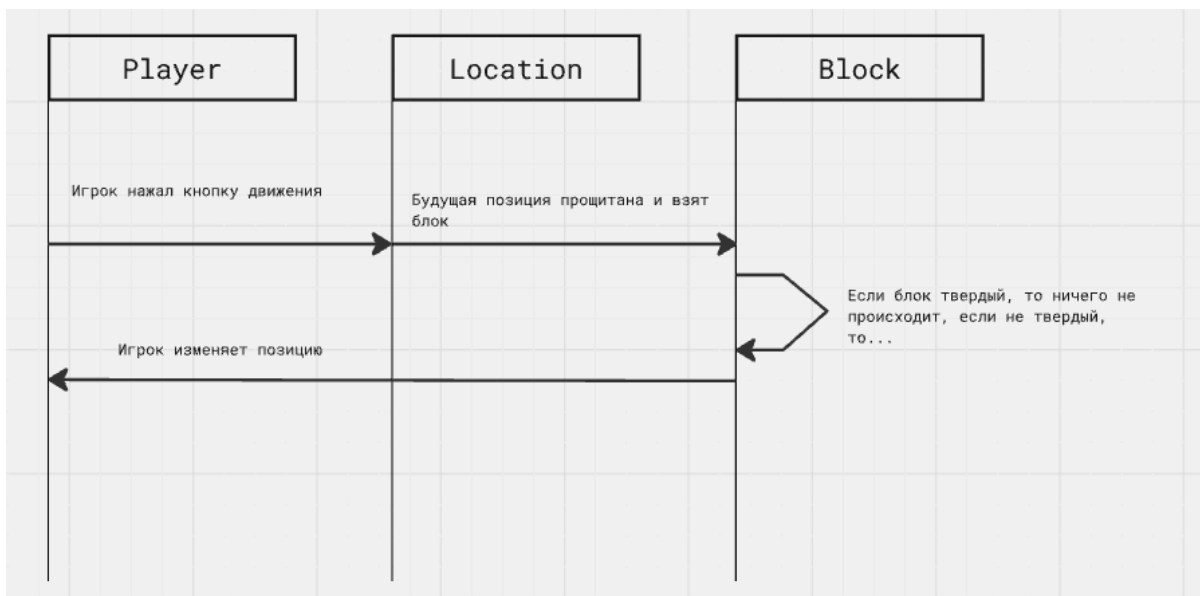
1. Остров делится на 16 частей
2. Каждая часть получает случайный биом, кроме тех, что заскриптованы
 - a. Случайные части заполняются случайными блоками с помощью сглаженного шума
 - b. Заскриптованные части вставляются и дополняются случайными блоками
3. В каждой части появляются необходимые entity

Сценарии:

Когда игрок стреляет из посоха по противнику:



При движении игрока (или любой другой сущности):



Когда игрок пытается добыть блок:

